# Swift Coroutines with KMP-NativeCoroutines

Aleksander Jaworski @ FootballCo - 07.07.2023 Droidcon Berlin

# Aleksander Jaworski

- Android Developer @ FootballCo
- Blog **akjaw.com** (Kotlin Multiplatform and Testing)
- Twitter @akjaworski1



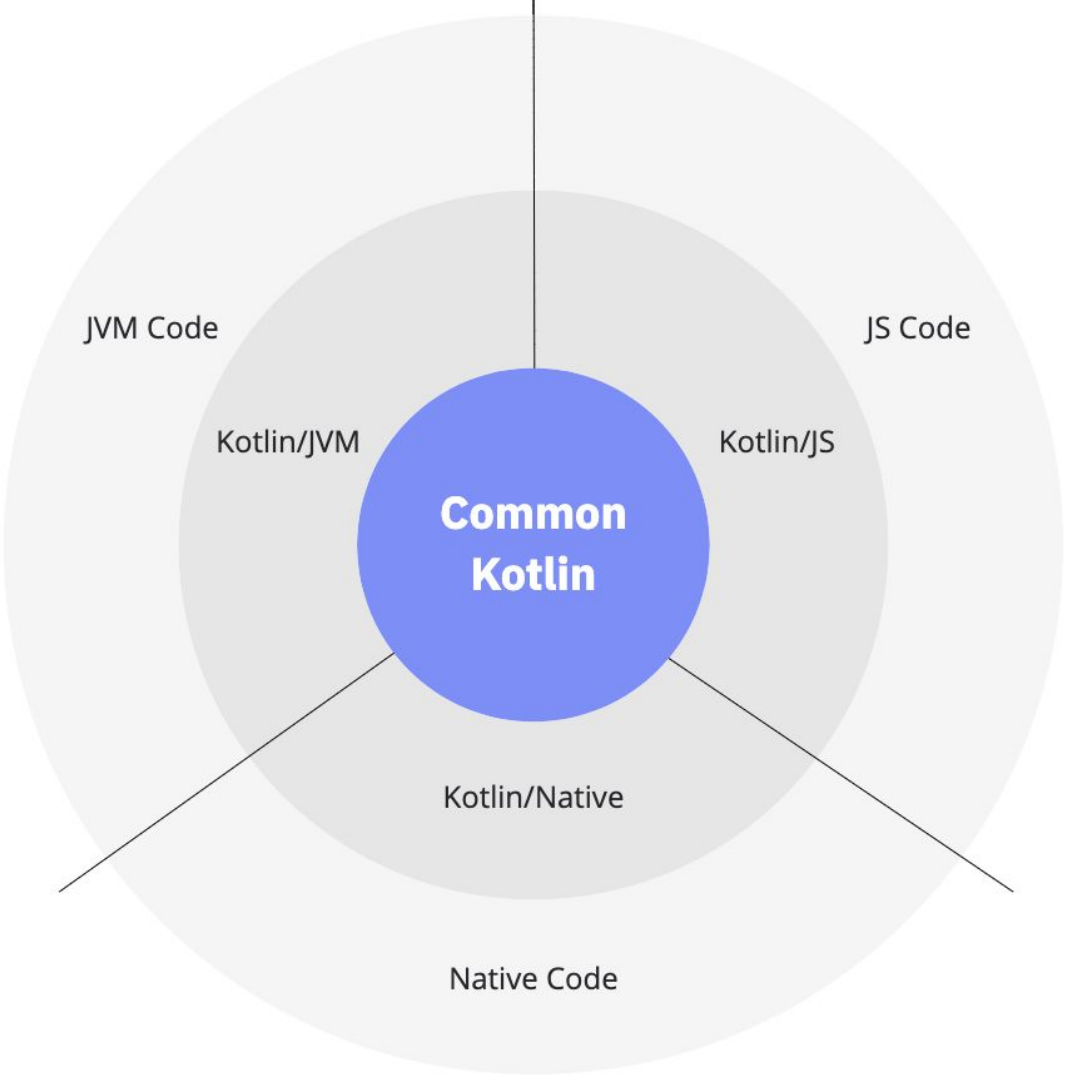
# What is Kotlin Multiplatform?

# What is Kotlin Multiplatform?

You probably know

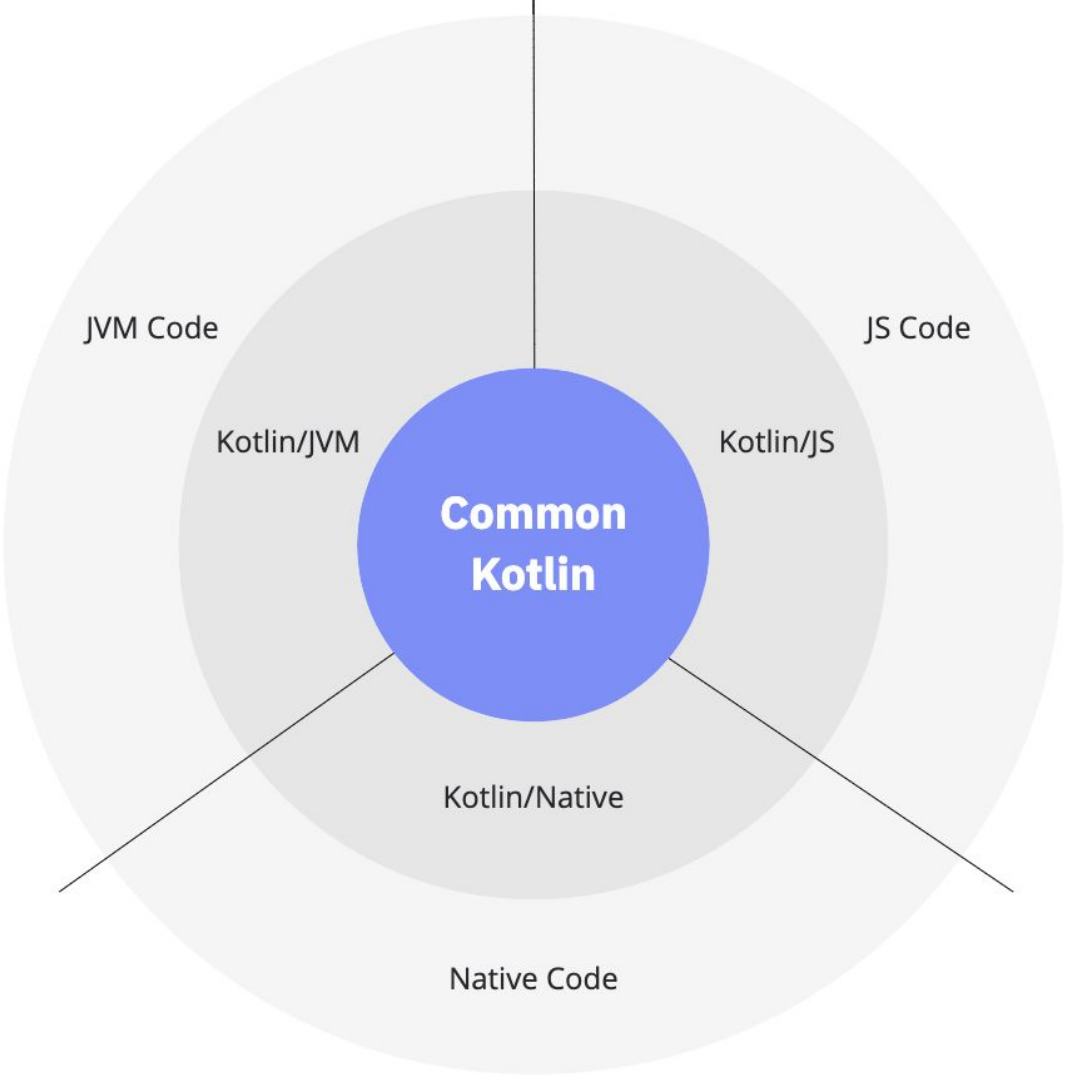
# What is Kotlin Multiplatform?

- One codebase, different platforms (iOS, Desktop, Web)



# What is Kotlin Multiplatform?

- One codebase, different platforms (iOS, Desktop, Web)
- Kotlin has the logic,platform has the UI*




* Compose Multiplatform is a thing

# How to call Kotlin from Swift?

# How to call Kotlin from Swift?

- Just call the generated Kotlin Multiplatform framework

# How to call Kotlin from Swift?

- Just call the generated Kotlin Multiplatform framework

```
// Kotlin
fun execute() {
   print("Exe")
}
```

# How to call Kotlin from Swift?

- Just call the generated Kotlin Multiplatform framework

```
// Kotlin
fun execute() {
   print("Exe")
}
```

```
// Generated
open func execute()
```

# How to call Kotlin from Swift?

- Just call the generated Kotlin Multiplatform framework

```kotlin
// Kotlin
fun execute() {
    print("Exe")
}
```

```
// Generated
open func execute()
```

```
// Swift
execute()
```

# How to call coroutines from Swift?

# How to call coroutines from Swift?

- It’s not so simple...

# How to call coroutines from Swift?

- It’s not so simple...

```
// Kotlin
suspend fun execute() {
   print("Exe")
}
```

# How to call coroutines from Swift?

- It’s not so simple...

```
// Kotlin
suspend fun execute() {
    print("Exe")
}
```

```
// Generated
 * @note This method converts instances of
CancellationException to errors.
 * Other uncaught Kotlin exceptions are
fatal.
open func execute(completionHandler:

@escaping (Error?) -> Void)
```

# How to call coroutines from Swift?

- It's not so simple...

```
// Kotlin
suspend fun execute() {
    print("Exe")
}
```

```
// Generated
 * @note This method converts instances of
CancellationException to errors.
 * Other uncaught Kotlin exceptions are
fatal.
open func execute(completionHandler:
@escaping (Error?) -> Void)
```

```
// Swift
execute { error in
    print(error)
}
```

# How to call coroutines from Swift?

- It’s not so simple...

```kotlin
// Kotlin
suspend fun execute() {
    print("Exe")
}
```

- How to cancel?

```
// Generated
 * @note This method converts instances of
CancellationException to errors.
 * Other uncaught Kotlin exceptions are
fatal.
open func execute(completionHandler:

@escaping (Error?) -> Void)
```

```
// Swift
execute { error in
    print(error)
}
```

# How to call coroutines from Swift?

- It’s not so simple...

```
// Kotlin
suspend fun execute() {
    print("Exe")
}
```

- How to cancel?
- Changing threads?

```
// Generated
 * @note This method converts instances of
CancellationException to errors.
 * Other uncaught Kotlin exceptions are
fatal.
open func execute(completionHandler:

@escaping (Error?) -> Void)
```

```
// Swift
execute { error in
    print(error)
}
```

# How to call coroutines from Swift?

- It’s not so simple...

```
// Kotlin
suspend fun execute() {
    print("Exe")
}
```

- How to cancel?
- Changing threads?
- Exception handling?

```
// Generated
 * @note This method converts instances of
CancellationException to errors.
 * Other uncaught Kotlin exceptions are
fatal.
open func execute(completionHandler:
@escaping (Error?) -> Void)
```

```
// Swift
execute { error in
    print(error)
}
```

# Agenda

- Calling default Suspend and Flows directly from Swift
- Creating Coroutine Adapters
- Using KMP-NativeCoroutines

# Agenda

- Calling default Suspend and Flows directly from Swift
- Creating Coroutine Adapters
- Using KMP-NativeCoroutines
- Exception Handling
- Cancellation

# Agenda

- Calling default Suspend and Flows directly from Swift
- Creating Coroutine Adapters
- Using KMP-NativeCoroutines
- Exception Handling
- Cancellation
- SwiftUI integrations*

# Why am I talking about this?

# Why am I talking about this?

- It’s important

# Why am I talking about this?

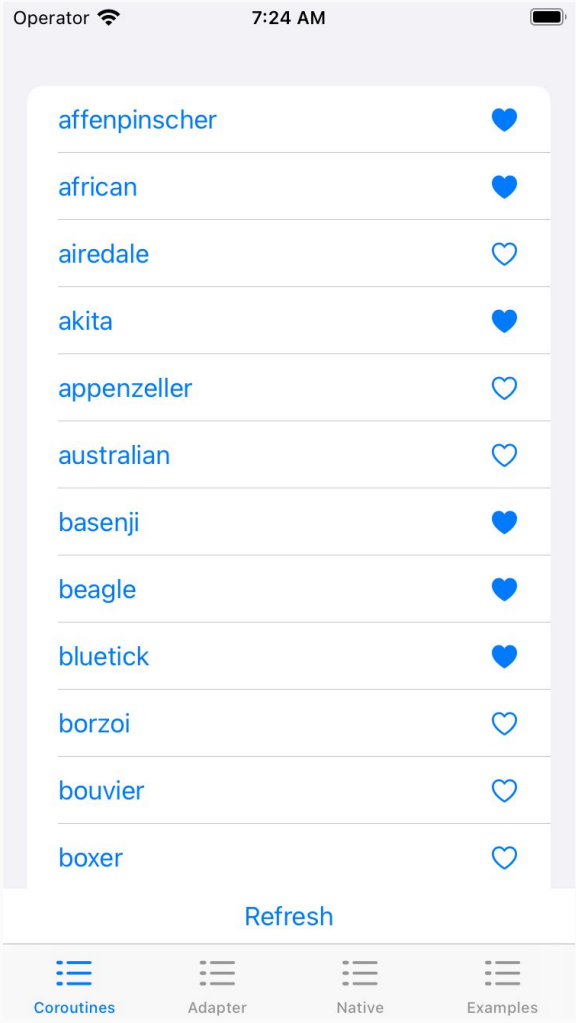
- It’s important
- When trying out Kotlin Multiplatform, we have to convince the iOS team

# Why am I talking about this?

- It’s important
- When trying out Kotlin Multiplatform, we have to convince the iOS team
- One of the biggest hurdles come when connecting Swift and Kotlin

## The App

- KaMPKit Starter from Touchlab
- Downloads from API i and saves to a Database
- 3 Coroutines call implementations



## The App

- KaMPKit Starter from Touchlab
- Downloads from API i and saves to a Database
- 3 Coroutines call implementations



github.com/AKJAW/Swift-Coroutines-With-KMP-NativeCoroutines

## What are we calling

- Ordinary function

```
class BreedViewModel() : ViewModel() {

    fun updateBreedFavorite(breed: Breed): Job
}
```

## What are we calling

- Ordinary function
- Suspend function

```
class BreedViewModel() : ViewModel() {


    suspend fun refreshBreeds(): Boolean

    fun updateBreedFavorite(breed: Breed): Job
}
```

## What are we calling

- Ordinary function
- Suspend function
- Data stream

```
class BreedViewModel() : ViewModel() {

    val breedState: StateFlow<BreedViewState>

    suspend fun refreshBreeds(): Boolean

    fun updateBreedFavorite(breed: Breed): Job
}
```

## What are we calling

- Ordinary function
- Suspend function
- Data stream

```
class BreedViewModel() : ViewModel() {

    val breedState: StateFlow<BreedViewState>

    suspend fun refreshBreeds(): Boolean

    fun updateBreedFavorite(breed: Breed): Job
}

actual abstract class ViewModel {

    actual val viewModelScope = MainScope()

    fun clear() {
        viewModelScope.coroutineContext
         .cancelChildren()
    }
}
```

# The easiest one is the Ordinary function




```
fun updateBreedFavorite(breed: Breed): Job {
    return viewModelScope.launch {
        breedRepository.updateBreedFavorite(breed)
    }
}
```

# The easiest one is the Ordinary function




```
fun updateBreedFavorite(breed: Breed): Job {
    return viewModelScope.launch {
        breedRepository.updateBreedFavorite(breed)
    }
}
```

```
func onBreedFavorite(_ breed: Breed) {
    viewModel?.updateBreedFavorite(breed: breed)
}
```

## Ordinary functions

- Fire and forget
- Modify internal state
- CoroutineScope is available

```
fun updateBreedFavorite(breed: Breed): Job {
    return viewModelScope.launch {
        breedRepository.updateBreedFavorite(breed)
    }
}
```

```
func onBreedFavorite(_ breed: Breed) {
    viewModel?.updateBreedFavorite(breed: breed)
}
```

# Suspend functions

# Suspending functions in Swift

- Default generated framework
- Manual Adaptera / Wrapper
- KMP-NativeCoroutines Plugin

# Suspending functions

- We want to know when it completes
- The return value
- The ability to cancel it

# Suspending functions

- We want to know when it completes
- The return value
- The ability to cancel it

```
suspend fun refreshBreeds(): Boolean {
    return try {
        breedRepository.refreshBreeds()
        true
    } catch (exception: Exception) {
        handleBreedError(exception)
        false
    }
}
```

## Generated function

```
viewModel?.refreshBreeds { wasRefreshed, error in
  print("refresh: \(wasRefreshed), \(error)")
}
```

## Generated function

- Forces an Optional value
- Cannot be cancelled

```
viewModel?.refreshBreeds { wasRefreshed, error in
  print("refresh: \(wasRefreshed), \(error)")
}
```

```
refresh: Optional(1), error: nil
```

## Generated function

- Forces an Optional value
- Cannot be cancelled
- **No error handling**

```
viewModel?.refreshBreeds { wasRefreshed, error in
  print("refresh: \(wasRefreshed), \(error)")
}
```

```
refresh: Optional(1), error: nil
```

```
/**
 * @note This method converts instances of CancellationException to errors.
 * Other uncaught Kotlin exceptions are fatal.
*/
- (void)refreshBreedsWithCompletionHandler:(void (^)(SharedBoolean * _Nullable, NSError * _Nullable))completionHandler __attribute__((swift_name("refreshBreeds(completionHandler:)")));
```

# Generated function developer experience

# Adapter / Wrapper - Kotlin

```
class SuspendAdapter<T : Any>(
    private val scope: CoroutineScope,
    private val suspender: suspend () -> T
) {

    fun subscribe(
        onSuccess: (item: T) -> Unit,
        onThrow: (error: Throwable) -> Unit
    ) = scope.launch {
        try {
            onSuccess(suspender())
        } catch (error: Throwable) {
            onThrow(error)
        }
    }

}
```

# Adapter / Wrapper - Kotlin

```kotlin
class SuspendAdapter<T : Any>(
    private val scope: CoroutineScope,
    private val suspender: suspend () -> T
) {

  fun subscribe(
      onSuccess: (item: T) -> Unit,
      onThrow: (error: Throwable) -> Unit
  ) = scope.launch {
      try {
          onSuccess(suspender())
      } catch (error: Throwable) {
          onThrow(error)
      }
  }

}
```

```kotlin
fun refreshBreeds() =
  SuspendAdapter(viewModel.scope) {
    viewModel.refreshBreeds()
  }
```

# Adapter / Wrapper - Kotlin

```kotlin
class SuspendAdapter<T : Any>(
    private val scope: CoroutineScope,
    private val suspender: suspend () -> T
) {

 fun subscribe(
     onSuccess: (item: T) -> Unit,
     onThrow: (error: Throwable) -> Unit
 ) = scope.launch {
     try {
         onSuccess(suspender())
     } catch (error: Throwable) {
         onThrow(error)
     }
 }
}
```

```kotlin
fun refreshBreeds() =
  SuspendAdapter(viewModel.scope) {
    viewModel.refreshBreeds()
  }
```

## Adapter/Wrapper - Swift

- Better error handling
- No Optional value
- Additional boilerplate

```
viewModel.refreshBreeds().subscribe(
    onSuccess: { value in
        print("completion \(value)")
    },
    onThrow: { error in
        print("error \(error)")
    }
)
```

## Adapter/Wrapper - Swift

- Better error handling
- No Optional value
- Additional boilerplate
- Ability to integrate with Combine or other

```
viewModel.refreshBreeds().subscribe(
    onSuccess: { value in
        print("completion \(value)")
    },
    onThrow: { error in
        print("error \(error)")
    }
)
```

```
createFuture(suspendAdapter: adapter)
    .sink { completion in
        print("completion \(completion)")
    } receiveValue: { value in
        print("recieveValue \(value)")
    }.store(in: &cancellables)
```

# Adapter/Wrapper developer experience

# KMP-NativeCoroutines

# Suspend KMP-NativeCoroutines

```
@NativeCoroutines
suspend fun nativeRefreshBreeds(): Boolean =
    refreshBreeds()
```

## NativeCoroutines

- No boilerplate

```
@NativeCoroutines
suspend fun nativeRefreshBreeds(): Boolean =
    refreshBreeds()
```

## NativeCoroutines

- No boilerplate
- No Optional
- **Dedicated Swift packages** Combine, Async, RxSwift

```
@NativeCoroutines
suspend fun nativeRefreshBreeds(): Boolean =
    refreshBreeds()
```

```
let suspend = viewModel.nativeRefreshBreeds()

    let value =
        try await asyncFunction(for: suspend)
    print("Async Success: \(value)")
```

## NativeCoroutines

- No boilerplate
- No Optional
- Dedicated Swift packages Combine, Async, RxSwift
- **Error handling**

```
@NativeCoroutines
suspend fun nativeRefreshBreeds(): Boolean =
    refreshBreeds()
```

```
let suspend = viewModel.nativeRefreshBreeds()
do {
    let value =
        try await asyncFunction(for: suspend)
    print("Async Success: \(value)")
} catch {
    print("Async Failed with error: \(error)")
}
```

## NativeCoroutines

- No boilerplate
- No Optional
- Dedicated Swift packages Combine, Async, RxSwift
- Error handling
- **Automatic cancelling**

```
@NativeCoroutines
suspend fun nativeRefreshBreeds(): Boolean =
    refreshBreeds()
```

```
func refresh() async {
  let suspend = viewModel.nativeRefreshBreeds()
  do {
      let value =
          try await asyncFunction(for: suspend)
      print("Async Success: \(value)")
  } catch {
      print("Async Failed with error: \(error)")
  }
}
```

# NativeCoroutines - asyncFunction

```
func refresh() async {
  let suspend = viewModel.nativeRefreshBreeds()
  do {
      let value =
          try await asyncFunction(for: suspend)
      print("Async Success: \(value)")
  } catch {
      print("Async Failed with error: \(error)")
  }
}
```

# NativeCoroutines - asyncResult

```
func refresh() async {
  let suspend = viewModel.nativeRefreshBreeds()
  let value = await asyncResult(for: suspend)
  switch value {
  case .success(let result):
      print("Async Success: \(result)")
  case .failure(let error):
      print("Async Failed with error: \(error)")
  }
}
```

# NativeCoroutines - asyncResult

```
func refresh() async {
  let suspend = viewModel.nativeRefreshBreeds()
  let value = await asyncResult(for: suspend)
  if case .success(let result) = value {
      print("Async Success: \(result)")
  }
}
```

# NativeCoroutines developer experience

# Flow Data Streams

# Data streams

- Generated framework - Many limitations + Boilerplate
- Adapter / Wrapper - Additional boilerplate
- KMP-NativeCoroutines

# Flow Generated framework

```
class Collector<T>:
Kotlinx_coroutines_coreFlowCollector {
    private let callback: (T) -> Void

    init(callback: @escaping (T) -> Void) {
        self.callback = callback
    }

    func emit(value: Any?, completionHandler:
@escaping (Error?) -> Void) {
        callback(value as! T)
        completionHandler(nil)
    }
}
```

# Flow Generated framework

```
class Collector<T>:
Kotlinx_coroutines_coreFlowCollector {
    private let callback: (T) -> Void

    init(callback: @escaping (T) -> Void) {
        self.callback = callback
    }

    func emit(value: Any?, completionHandler:
@escaping (Error?) -> Void) {
        callback(value as! T)
        completionHandler(nil)
    }
}
```

```
viewModel.breedState.collect(
    collector: Collector<BreedViewState> {
        [weak self] dogsState in

        self?.loading = dogsState.isLoading
        self?.breeds = dogsState.breeds
        self?.error = dogsState.error
    },
    completionHandler: { error in
        print("breed collection completion
error: \(error)")
    }
)
```

# Flow Adapter / Wrapper

```
class FlowAdapter<T : Any>(
 private val scope: CoroutineScope,
 private val flow: Flow<T>
) {
 fun subscribe(
   onEach: (item: T) -> Unit,
   onComplete: () -> Unit,
   onThrow: (error: Throwable) -> Unit
 ): Canceller = JobCanceller(
   flow.onEach { onEach(it) }
     .catch { onThrow(it) }
     .onCompletion { onComplete() }
     .launchIn(scope)
 )
}

interface Canceller {
 fun cancel()
}

private class JobCanceller(private val job:
Job) : Canceller {
 override fun cancel() {
   job.cancel()
 }
}
```

# Flow Adapter / Wrapper

```
class FlowAdapter<T : Any>(
 private val scope: CoroutineScope,
 private val flow: Flow<T>
) {
 fun subscribe(
   onEach: (item: T) -> Unit,
   onComplete: () -> Unit,
   onThrow: (error: Throwable) -> Unit
 ): Canceller = JobCanceller(
   flow.onEach { onEach(it) }
     .catch { onThrow(it) }
     .onCompletion { onComplete() }
     .launchIn(scope)
 )
}

interface Canceller {
 fun cancel()
}

private class JobCanceller(private val job:
Job) : Canceller {
 override fun cancel() {
   job.cancel()
 }
}
```

```
viewModel.breeds.subscribe(
    onEach: { [weak self] dogsState in
        self?.loading = dogsState.isLoading
        self?.breeds = dogsState.breeds
        self?.error = dogsState.error
    },
    onComplete: { print("Subscription end") },
    onThrow: { error in  print("Subscription
error: \(error)") }
)
```

# Flow KMP-NativeCoroutines

```
@NativeCoroutinesState
val nativeBreedState: StateFlow<BreedViewState> =
    breedState
```

# Flow KMP-NativeCoroutines

```
private val mutableBreedState =
    MutableStateFlow(
        BreedViewState(isLoading = true)
    )

@NativeCoroutinesState
val nativeBreedState: StateFlow<BreedViewState> =
    breedState
```

## Native Flow

- Integration with Native solutions

```
let nativeFlow = viewModel.nativeBreedStateFlow

  let sequence = asyncSequence(for: nativeFlow)
```

## Native Flow

- Integration with Native solutions
- **No casting needed**

```
let nativeFlow = viewModel.nativeBreedStateFlow

  let sequence = asyncSequence(for: nativeFlow)
  for try await dogsState in sequence {
      self.loading = dogsState.isLoading
      self.breeds = dogsState.breeds
      self.error = dogsState.error
  }
```

## Native Flow

- Integration with Native solutions
- No casting needed

```
let nativeFlow = viewModel.nativeBreedStateFlow
do {
  let sequence = asyncSequence(for: nativeFlow)
  for try await dogsState in sequence {
      self.loading = dogsState.isLoading
      self.breeds = dogsState.breeds
      self.error = dogsState.error
  }
} catch {
  print("Async Failed with error: \(error)")
}
```

## Native Flow

- Integration with Native solutions
- No casting needed
- **Automatic cancellation**

```
func activate() async {
  let nativeFlow = viewModel.nativeBreedStateFlow
  do {
    let sequence = asyncSequence(for: nativeFlow)
    for try await dogsState in sequence {
        self.loading = dogsState.isLoading
        self.breeds = dogsState.breeds
        self.error = dogsState.error
    }
  } catch {
    print("Async Failed with error: \(error)")
  }
}
```

## Native Flow

- Integration with Native solutions
- No casting needed
- Automatic cancellation
- Collection on **Default dispatcher**

```
@MainActor
func activate() async {
  let nativeFlow = viewModel.nativeBreedStateFlow
  do {
    let sequence = asyncSequence(for: nativeFlow)
    for try await dogsState in sequence {
        self.loading = dogsState.isLoading
        self.breeds = dogsState.breeds
        self.error = dogsState.error
    }
  } catch {
    print("Async Failed with error: \(error)")
  }
}
```

# Controlling the Scope

- Ability to cancel directly from Kotlin
- Specify which thread should the suspension happen

```
@NativeCoroutineScope
actual val viewModelScope =
    MainScope()
```

# Controlling the Scope

- Ability to cancel directly from Kotlin
- Specify which thread should the suspension happen
- MainActor still required

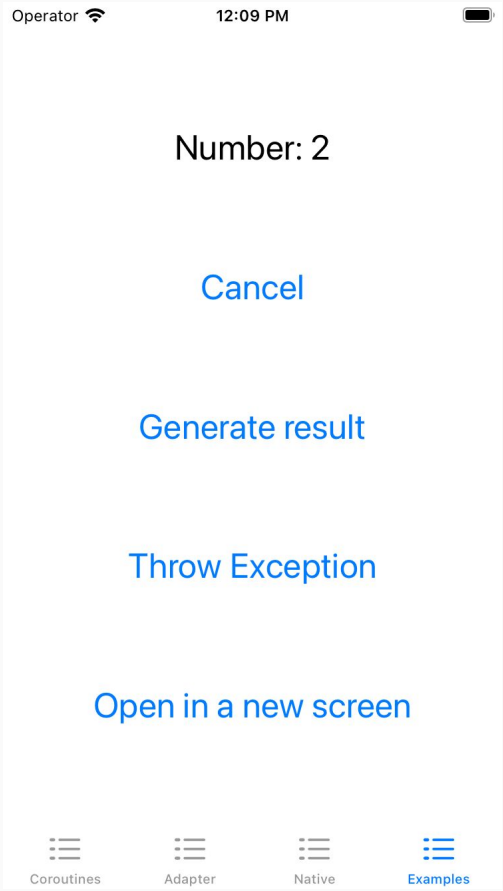
```
@NativeCoroutineScope
actual val viewModelScope =
    MainScope()

@MainActor
func activate() async {

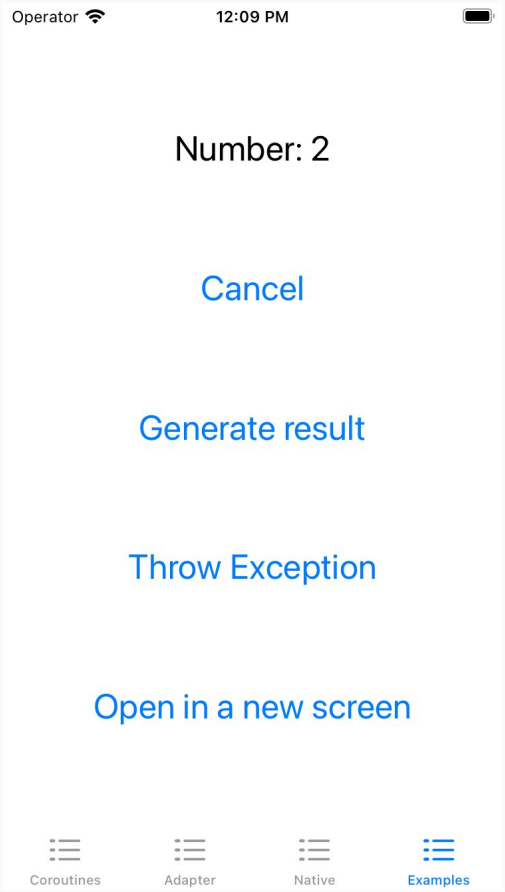
}
```

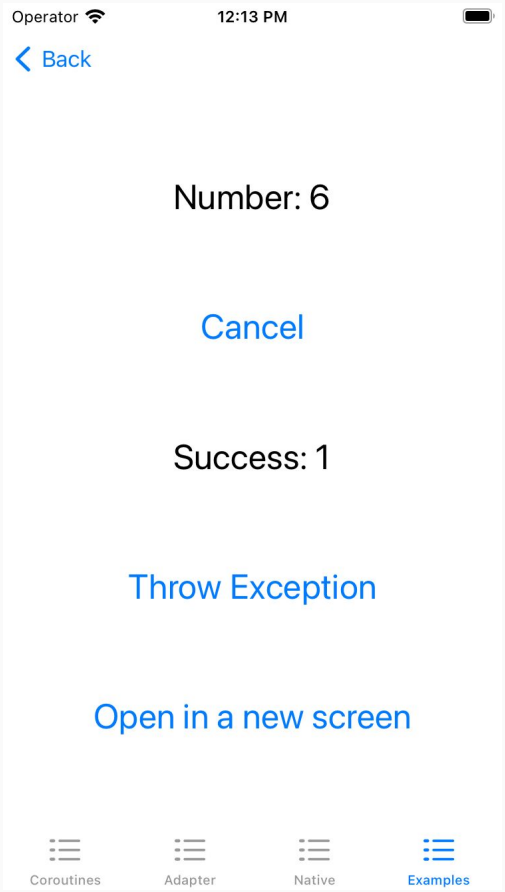
# Playground Screen

# Playground

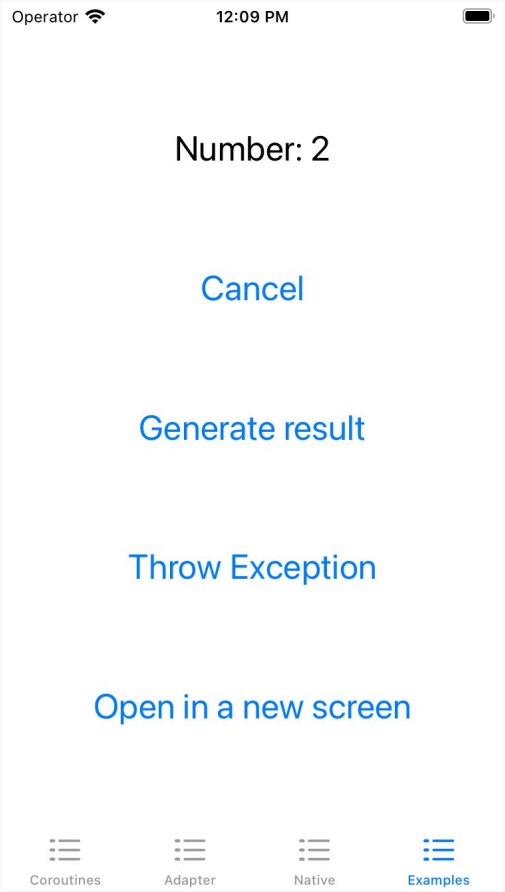

Operator
12:09 PM
Number: 2
Cancel
Generate result
Throw Exception
Open in a new screen
Coroutines
Adapter
Native
Examples

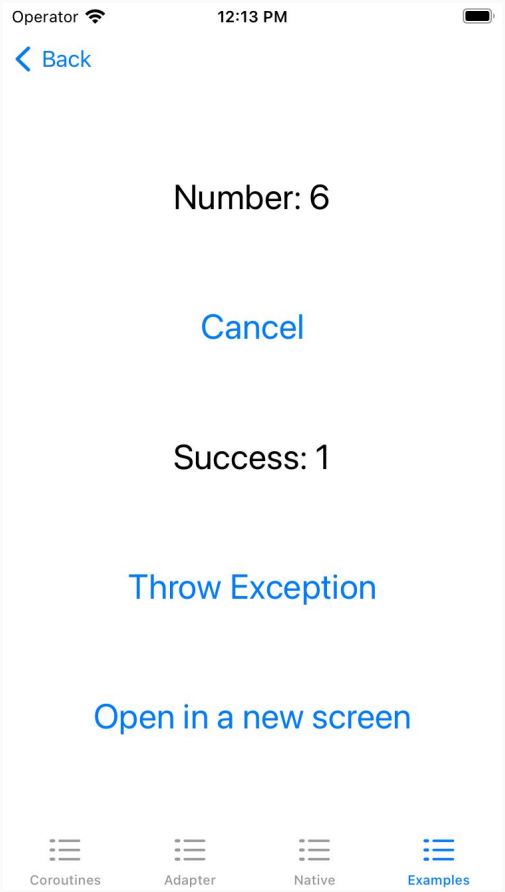
# Playground

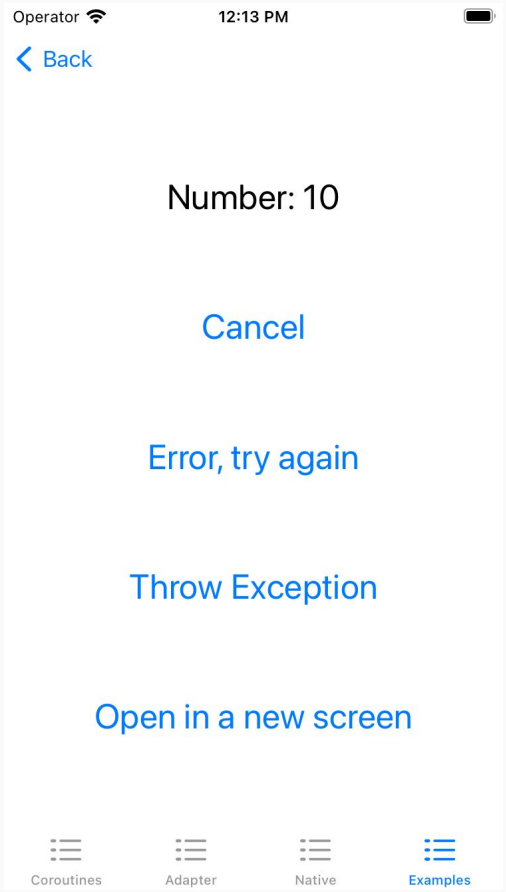

Operator
12:09 PM
Number: 2
Cancel
Generate result
Throw Exception
Open in a new screen
Coroutines
Adapter
Native
Examples


Operator
12:13 PM
Back
Number: 6
Cancel
Success: 1
Throw Exception
Open in a new screen
Coroutines
Adapter
Native
Examples

# Playground


Operator
12:09 PM
Number: 2
Cancel
Generate result
Throw Exception
Open in a new screen
Coroutines
Adapter
Native
Examples


Operator
12:13 PM
Back
Number: 6
Cancel
Success: 1
Throw Exception
Open in a new screen
Coroutines
Adapter
Native
Examples


Operator
12:13 PM
Back
Number: 10
Cancel
Error, try again
Throw Exception
Open in a new screen
Coroutines
Adapter
Native
Examples

# Suspend Exception Handling

```
@NativeCoroutines
suspend fun throwException() {
    delay(1000)
    throw IllegalStateException()
}
```

# Suspend Exception Handling

```
print("async exception start")
do {
    let result = try await asyncFunction(for: suspend)
    print("async exception success \(result)")
} catch {
    print("async exception failure \(error)")
}
```

# Suspend Exception Handling

```
print("async exception start")
do {
    let result = try await asyncFunction(for: suspend)
    print("async exception success \(result)")
} catch {
    print("async exception failure \(error)")
}
```

```
18:07:03 async exception start
18:07:04 async exception failure
(ErrorDomain=KotlinExcepton Code=0 "(null)"
UserInfo={KotlinException=kotlin.IllegalStateException})
```

# Suspend Exception Handling - Variant

```
print("async exception start")
let result = await asyncResult(for: suspend)
switch result {
case .success(let value):
    print("async exception success \(value)")
case .failure(let error):
    print("async exception failure \(error)")
}
```

```
18:07:53 async exception start
18:07:54 async exception failure
(ErrorDomain=KotlinExcepton Code=0 "(null)"
UserInfo={KotlinException=kotlin.IllegalStateException})
```

# Flow Exception Handling

```
@NativeCoroutines
val errorFlow: Flow<Int> = flow {
    repeat(3) { number ->
        emit(number)
        delay(1000)
    }
    throw IllegalStateException()
}
```

# Flow Exception Handling

```
do {
    let sequence = asyncSequence(for: viewModel.errorFlow)
    for try await number in sequence {
        print("sequence exception n: \(number)")
    }
} catch {
    print("sequence exception error: \(error)")
}
```

# Flow Exception Handling

```
do {
    let sequence = asyncSequence(for: viewModel.errorFlow)
    for try await number in sequence {
        print("sequence exception n: \(number)")
    }
} catch {
    print("sequence exception error: \(error)")
}
```

```
2023-06-28 18:14:24 sequence exception number 0
2023-06-28 18:14:25 sequence exception number 1
2023-06-28 18:14:26 sequence exception number 2
2023-06-28 18:14:27 sequence exception error
(Error Domain=KotlinException Code=0 "(null)"
UserInfo={KotlinException=kotlin.IllegalStateException})
```

# Flow Exception Handling - Variant

```
let sequence = asyncSequence(for: viewModel.errorFlow)
for try await number in sequence {
    print("sequence exception n: \(number)")
}
```

# Flow Exception Handling - Variant

```
func throwException() async throws {
    let sequence = asyncSequence(for: viewModel.errorFlow)
    for try await number in sequence {
        print("sequence exception n: \(number)")
    }
}
```

# Flow Exception Handling - Variant

```
func throwException() async throws {
    let sequence = asyncSequence(for: viewModel.errorFlow)
    for try await number in sequence {
        print("sequence exception n: \(number)")
    }
}
```

```
try? await observableModel.throwException()
```

# Flow Exception Handling - Variant

```
func throwException() async throws {
    let sequence = asyncSequence(for: viewModel.errorFlow)
    for try await number in sequence {
        print("sequence exception n: \(number)")
    }
}
```

```
try? await observableModel.throwException()
```

```
2023-06-28 18:15:24 sequence exception number 0
2023-06-28 18:15:25 sequence exception number 1
2023-06-28 18:15:26 sequence exception number 2
```

## Exception Handling

- Exception handling is awkward in Swift

```
@NativeCoroutines
suspend fun throwException() {
    delay(1000)
    throw IllegalStateException()
}
```

```
@NativeCoroutines
val errorFlow: Flow<Int> = flow {
    repeat(3) { number ->
        emit(number)
        delay(1000)
    }
    throw IllegalStateException()
}
```

## Exception Handling

- Exception handling is awkward in Swift
- Better to return an error value

```
@NativeCoroutines
suspend fun throwException() {
    delay(1000)
    throw IllegalStateException()
}
```

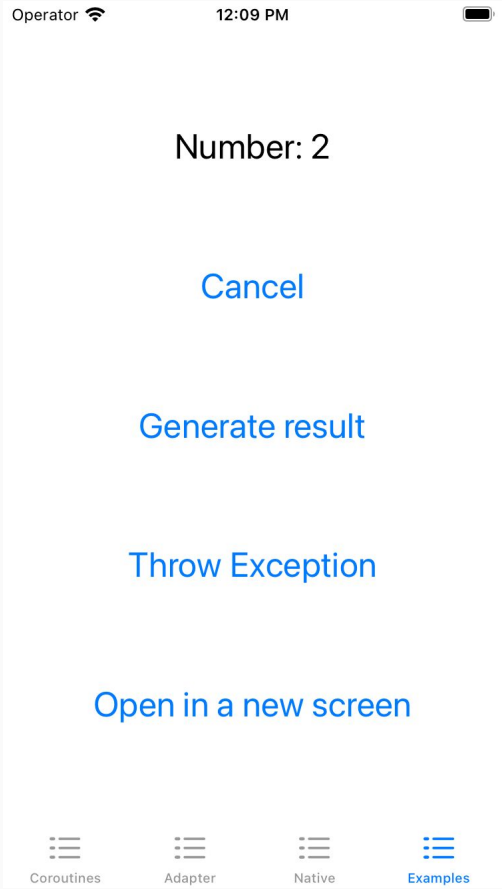
```
@NativeCoroutines
val errorFlow: Flow<Int> = flow {
    repeat(3) { number ->
        emit(number)
        delay(1000)
    }
    throw IllegalStateException()
}
```

# Cancellation

```
@NativeCoroutines
val numberFlow: Flow<Int> = flow {
    var i = 0
    while (true) {
        emit(i++)
        delay(1000)
    }
}.onEach { number ->
    log.i("numberFlow onEach: $number")
}.onCompletion { throwable ->
    log.i("numberFlow onCompletion: $throwable")
}
```

# Manual cancellation

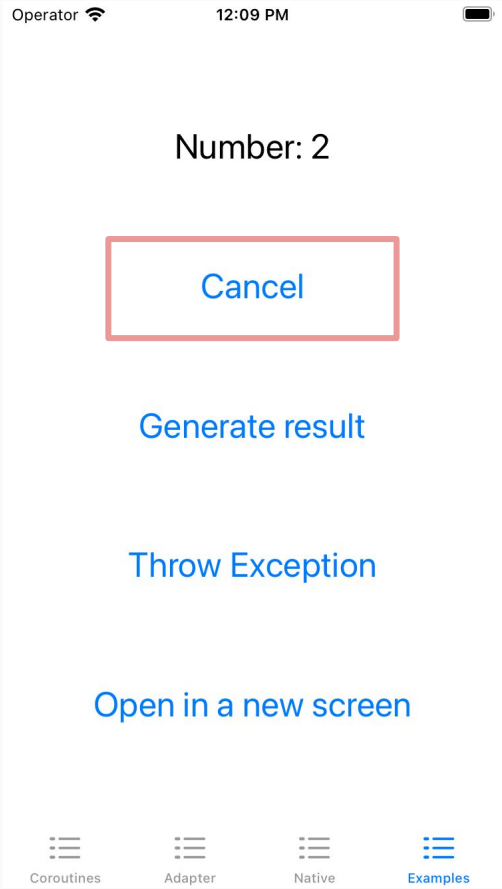
```
func listenToNumbers() async {
  numberTask = Task {
      let sequence =
        asyncSequence(for: viewModel.numberFlow)
      for try await number in sequence {
        self.number = number.intValue
      }
  }
}
```



# Manual cancellation

```
func listenToNumbers() async {
  numberTask = Task {
      let sequence =
        asyncSequence(for: viewModel.numberFlow)
      for try await number in sequence {
        self.number = number.intValue
      }
  }
}

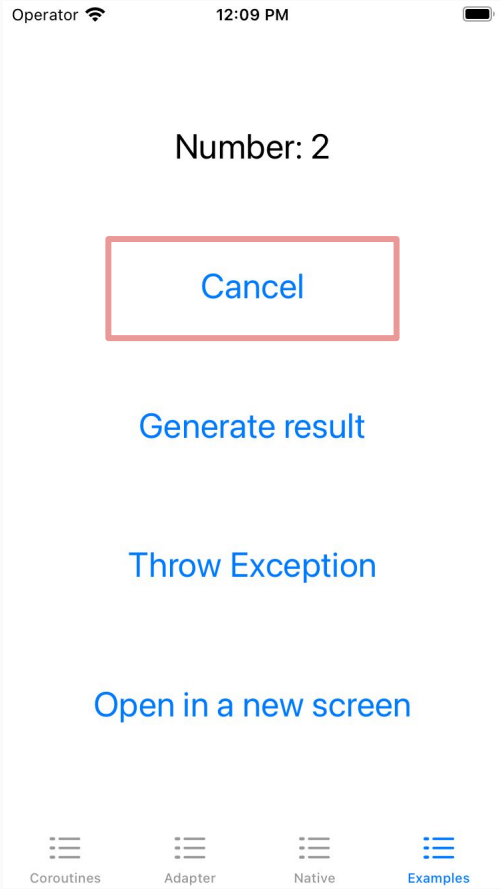
func cancel() {
    numberTask?.cancel()
    numberTask = nil
}
```



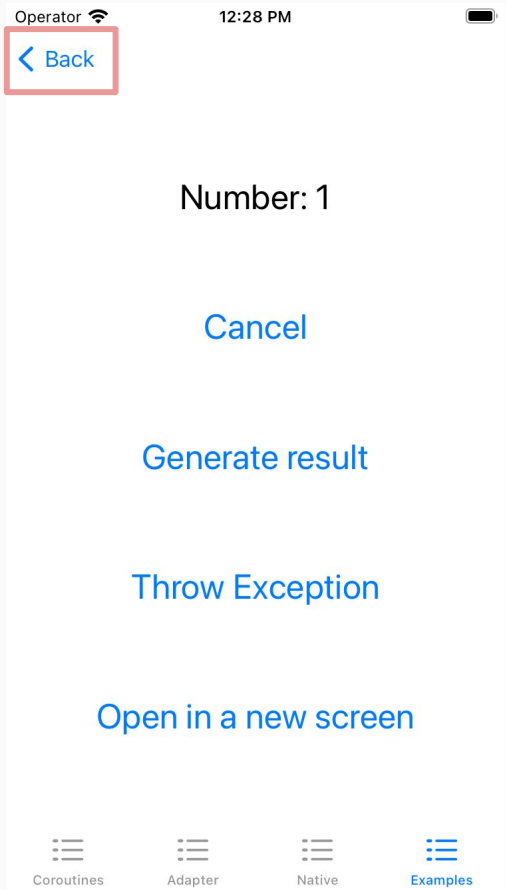
# Manual cancellation

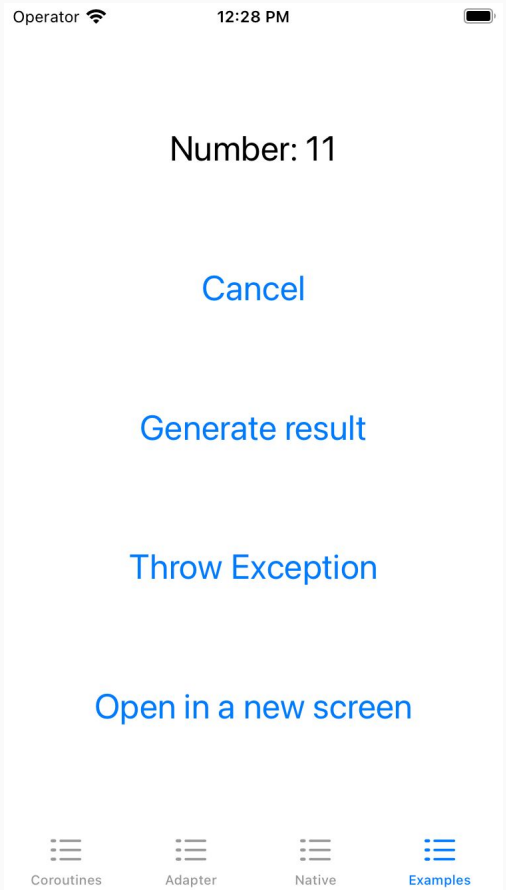
```
func listenToNumbers() async {
  numberTask = Task {
      let sequence =
        asyncSequence(for: viewModel.numberFlow)
      for try await number in sequence {
        self.number = number.intValue
      }
  }
}
func cancel() {
    numberTask?.cancel()
    numberTask = nil
}
```

```
2023-06-30 12:21:30 numberFlow onEach: 0
2023-06-30 12:21:31 numberFlow onEach: 1
2023-06-30 12:21:32 numberFlow onEach: 2
2023-06-30 12:21:33 numberFlow onCompletion:
kotlinx.coroutines.JobCancellationException: …
```



# Automatic cancellation



# Automatic cancellation

```
func listenToNumbers() async {
    let sequence =
        asyncSequence(for: viewModel.numberFlow)
    for try await number in sequence {
      self.number = number.intValue
    }
}
```

# Automatic cancellation

```
func listenToNumbers() async {
    let sequence =
        asyncSequence(for: viewModel.numberFlow)
    for try await number in sequence {
      self.number = number.intValue
    }
}

var body: some View {
    ...
}.task {
    await observableModel.listenToNumbers()
}
```

# Automatic cancellation

```
func listenToNumbers() async {
    let sequence =
        asyncSequence(for: viewModel.numberFlow)
    for try await number in sequence {
      self.number = number.intValue
    }
}
```

```
var body: some View {
    ...
}.task {
    await observableModel
            .listenToNumbers()
}
```

```
2023-03-24 12:28:13 init 0x00006000024be000
2023-03-24 12:28:13 numberFlow onEach: 0
2023-03-24 12:28:14 numberFlow onEach: 1
2023-03-24 12:28:15 numberFlow onEach: 2
2023-03-24 12:28:16 numberFlow onEach: 3
2023-03-24 12:28:16 deinit 0x00006000024be000
2023-03-24 12:28:16 numberFlow onCompletion:
kotlinx.coroutines.JobCancellationException: ...
```

# Cancellation

- Freeing up resources

```
func cancel() {
    numberTask?.cancel()
    numberTask = nil
}
```

# Cancellation

- Freeing up resources
- Avoiding unnecessary networking

```
func cancel() {
    numberTask?.cancel()
    numberTask = nil
}
```

# Cancellation

- Freeing up resources
- Avoiding unnecessary networking
- Avoiding Memory leaks

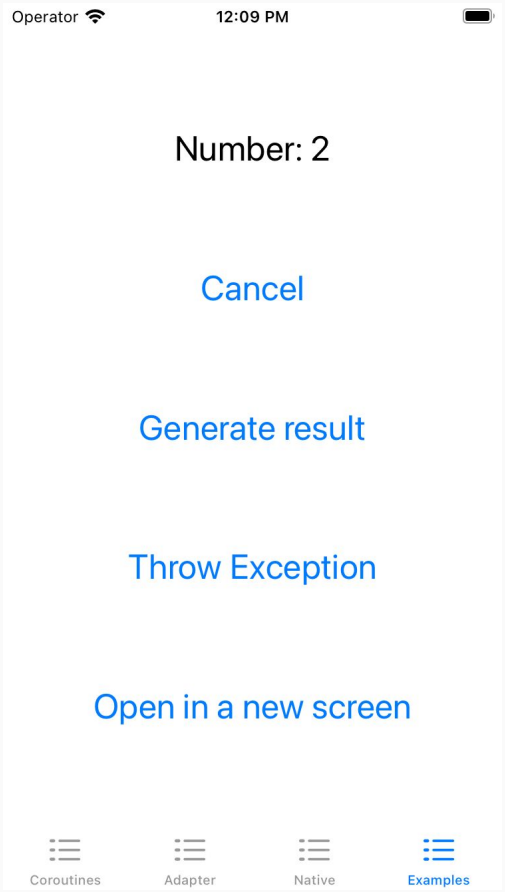
```
func cancel() {
    numberTask?.cancel()
    numberTask = nil
}
```

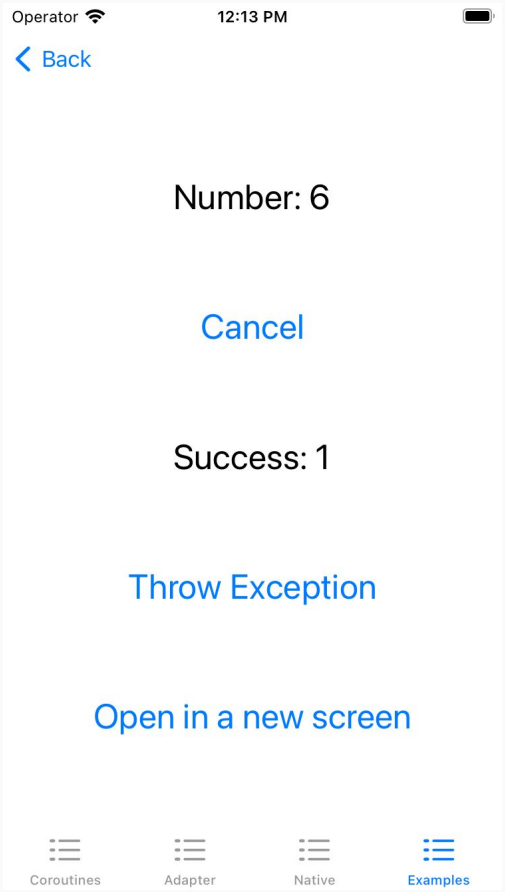
# SwiftUI

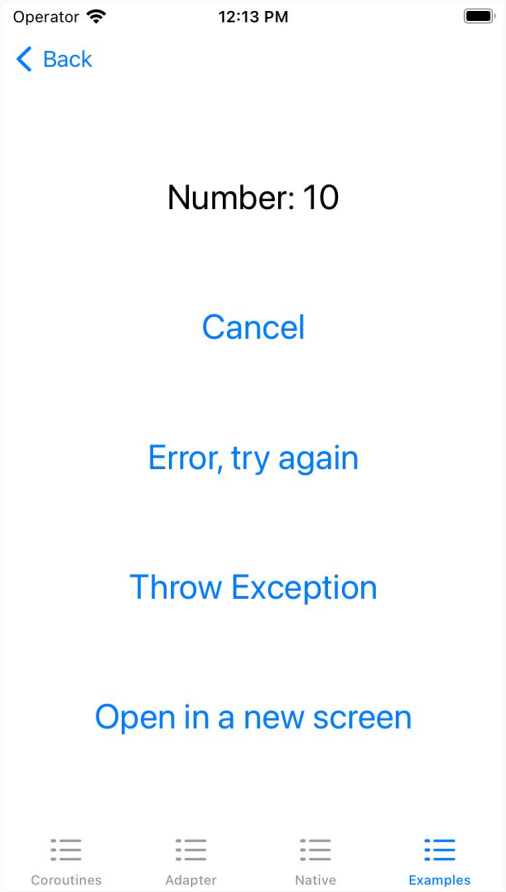
# SwiftUI Integration

- Async await
- ObservableObject
- @Published
- @StateObject / @ObservedObject

# Playground


Operator
12:09 PM
Number: 2
Cancel
Generate result
Throw Exception
Open in a new screen
Coroutines
Adapter
Native
Examples


Operator
12:13 PM
Back
Number: 6
Cancel
Success: 1
Throw Exception
Open in a new screen
Coroutines
Adapter
Native
Examples


Operator
12:13 PM
Back
Number: 10
Cancel
Error, try again
Throw Exception
Open in a new screen
Coroutines
Adapter
Native
Examples

# Published Int

```
@NativeCoroutines
val numberFlow: Flow<Int> = flow {
    var i = 0
    while (true) {
        emit(i++)
        delay(1000)
    }
}
```

# Published Int

```
private class CoroutinesExampleModel: ObservableObject {









}
```

# Published Int

```
private class CoroutinesExampleModel: ObservableObject {

  @Published
  var number: Int = -1









}
```

# Published Int

```
private class CoroutinesExampleModel: ObservableObject {

   @Published
   var number: Int = -1

   func listenToNumbers() async {
      do {
         let sequence = asyncSequence(for: viewModel.numberFlow)



      }


    }
}
```

# Published Int

```
private class CoroutinesExampleModel: ObservableObject {

    @Published
    var number: Int = -1

    func listenToNumbers() async {
        do {
            let sequence = asyncSequence(for: viewModel.numberFlow)
            for try await number in sequence {
                self.number = number.intValue
            }
        }


    }
}
```

# Published Int

```
private class CoroutinesExampleModel: ObservableObject {

   @Published
   var number: Int = -1

   func listenToNumbers() async {
      do {
         let sequence = asyncSequence(for: viewModel.numberFlow)
         for try await number in sequence {
             self.number = number.intValue
         }
      } catch {
         print("Async numberFlow Failed with error: \(error)")
      }
    }

}
```

# Published Int

```
struct CoroutinesExampleScreen: View {




}
```

# Published Int

```
struct CoroutinesExampleScreen: View {

  @StateObject
  private var observableModel = CoroutinesExampleModel()



}
```

# Published Int

```
struct CoroutinesExampleScreen: View {

  @StateObject
  private var observableModel = CoroutinesExampleModel()

  var body: some View {
      Text("Number: \(observableModel.number)")
  }
}
```

# Published Int

Number: 3

Number: 13

Number: 28

Number: 47

# KmmViewModel - Bonus

- The same author
- Generates ObservableObject itd.

# KmmViewModel - Bonus

- The same author
- Generates ObservableObject itd.

| ObservableObject | KMMViewModel |
| --- | --- |
| @StateObject | @StateViewModel |
| @ObservedObject | @ObservedViewModel |
| @EnvironmentObject | @EnvironmentViewModel |
| environmentObject(_:) | environmentViewModel(_:) |

# Summary

# What we've learned

- How to call Suspend and Flow from swift using different approaches

# What we've learned

- How to call Suspend and Flow from swift using different approaches
- How Kotlin Exceptions can affect the iOS App and how to handle them

# What we've learned

- How to call Suspend and Flow from swift using different approaches
- How Kotlin Exceptions can affect the iOS App and how to handle then
- Why Cancelling coroutines is a good idea and how to do it from Swift

# What we've learned

- How to call Suspend and Flow from swift using different approaches
- How Kotlin Exceptions can affect the iOS App and how to handle then
- Why Cancelling coroutines is a good idea and how to do it from Swift
- Integrating KMP-NativeCoroutines with SwiftUi*

# Learning Kotlin Multiplatform

- https://johnoreilly.dev/
- https://touchlab.co/blog/
- https://www.marcogomiero.com/
- https://akjaw.com/

# My final message

# My final message

KMP-NativeCoroutines makes the iOS developers life easier and more **fun**.

# Presentation Async



https://akjaw.com/assets/talks/NativeCoroutines-Async-Droidcon-Berlin.pdf

# Presentation Combine



https://akjaw.com/assets/talks/NativeCoroutines-Combine-Droidcon-Berlin.pdf

# Sources

- https://github.com/AKJAW/Swift-Coroutines-With-KMP-NativeCoroutines
- https://github.com/touchlab/KaMPKit
- https://github.com/rickclephas/KMP-NativeCoroutines
- https://github.com/rickclephas/KMM-ViewModel
- https://dev.to/touchlab/kotlin-coroutines-and-swift-revisited-j5h
- https://www.slideshare.net/ChristianMelchior/coroutines-for-kotlin-multiplatform-in-practise

# Questions?